TabulaVibe AI: Technical Reference

Reverse-engineered from the source on 2026-10-21 at commit `0b58f0a` (version 0.1.0); updated for the UI v2 redesign (0.2.0). Design-system details are in `DESIGN.md`. Every behaviour described here was read from the code. File references use `path:line`.

Contents

1. What the system is
2. System architecture
3. Repository map
4. Request lifecycle
5. Backend in depth - 5.1 Application factory, middleware and error model - 5.2 Configuration - 5.3 Ingestion - 5.4 Type inference and profiling - 5.5 Analysis engine - 5.6 Exports - 5.7 Optional AI narrative - 5.8 Request schemas - 5.9 Samples and the Lambda entry point
6. API reference
7. Frontend in depth
8. Design system and charts
9. Testing strategy
10. Build, tooling and developer workflow
11. Cloud architecture (AWS)
12. Security model
13. Performance and limits
14. Findings from the reverse-engineering pass
15. How to extend the system
16. Design decisions and trade-offs
17. Interview talking points
18. Glossary

---

1. What the system is

TabulaVibe AI is a **stateless analytics web application**. It turns one uploaded spreadsheet (CSV or XLSX) into five outputs:

1. a **profile**: inferred column types, quality issues, statistics and a preview;
2. a **KPI analysis**: aggregations, optionally grouped and filtered;
3. a **dashboard**: KPI cards, a chart chosen to suit the data, and a table;
4. **exports**: a results CSV, a cleaned-data CSV and a printable report;
5. an optional **AI narrative** describing the computed results.

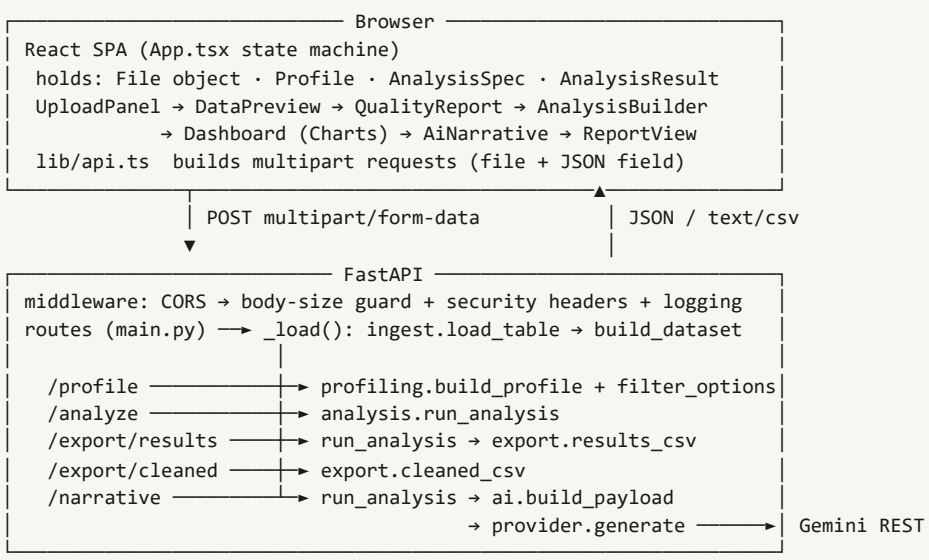
Three principles shape every module:

Principle	Where it shows up
Python is the single source of truth for numbers	`analysis.py` computes everything. The frontend only formats, and the LLM only describes
Report, never silently fix	Missing and invalid values are counted and excluded, never altered. Cleaning is opt-in and announced

Principle	Where it shows up
Store nothing	No database, sessions or file storage. Each request carries the file and the server forgets it afterwards

2. System architecture

2.1 Logical view



2.2 Deployment views

Environment	Frontend	Backend	Connection
Local development	Vite dev server (`:5173`)	Uvicorn (`:8000`)	Vite proxies `/api` to the backend, so the browser sees one origin and needs no CORS
End-to-end tests	Vite on `:5181`	Uvicorn on `:8011`	Started by the Playwright `webServer` config
AWS (prepared)	S3 (private) behind CloudFront with OAC	Lambda (python3.14, arm64) behind a Function URL via Mangum	Cross-origin. `VITE_API_BASE_URL` is set to the Function URL; the app's CORS allows only the CloudFront domain

2.3 Why stateless re-upload

The browser keeps the `File` object and sends it again with every request. Consequences:

- **No storage tier.** Nothing is persisted, so there is no retention or deletion policy to build, and "we don't store your data" is literally true.
- **Horizontally trivial.** Any Lambda instance can serve any request.
- **Cost:** the file is re-parsed on each call (sub-second within the limits) and re-transmitted, at most 4 MB.

3. Repository map

```
tabulavibe-ai/
├─ backend/
│  ├─ app/
│  │  ├─ __init__.py          version "0.1.0"
│  │  ├─ main.py              app factory, middleware, error handlers, routes
│  │  ├─ config.py            Settings (pydantic-settings, TABULAVIBE_* env vars)
│  │  ├─ errors.py            AppError / DataError (user-safe messages)
│  │  ├─ ingest.py            bytes → RawTable (all strings) with every safety check
│  │  ├─ profiling.py         RawTable → Dataset (typed), profile, label merging
│  │  ├─ analysis.py          AnalysisSpec → KPIs, groups, chart advice
│  │  ├─ export.py            CSV writers, formula-injection guard
│  │  ├─ ai.py                provider protocol, Gemini REST, rate limiter, payload
│  │  ├─ schemas.py           Measure, Filter, AnalysisSpec, CleanOptions
│  │  ├─ samples.py           sample registry
│  │  ├─ samples/             retail_sales.csv, service_bookings.xlsx (synthetic)
│  │  └─ lambda_handler.py    Mangum(app)
│  ├─ scripts/generate_samples.py   seeded generator for the samples
│  ├─ tests/                  pytest suite (83 functions → 108 cases)
│  ├─ requirements.txt / requirements-dev.txt / pyproject.toml
├─ frontend/
│  ├─ src/
│  │  ├─ App.tsx              workflow state machine
│  │  ├─ components/          UploadPanel, DataPreview, QualityReport, AnalysisBuilder,
│  │  │                       Dashboard, Charts, AiNarrative, ReportView, ui
│  │  ├─ lib/                 api, types, spec, format, charts
│  │  ├─ test/                setup + fixtures
│  │  └─ index.css            Tailwind import, design tokens, dark mode, print rules
│  ├─ e2e/journey.spec.ts     Playwright journeys
│  ├─ vite.config.ts / playwright.config.ts / tsconfig*.json / .oxlintrc.json
├─ infra/
│  ├─ template.yaml           AWS SAM template
│  ├─ stage_backend.py        copies runtime code to infra/build/api
│  └─ deploy.ps1              build → deploy → upload → smoke test
├─ docs/                      architecture, deployment, status, changelog, research, this file
├─ README.md · PROJECT_SPEC.md · CLAUDE.md · .env.example · .gitignore · .gitattributes
```

4. Request lifecycle (end to end)

Example: the user runs "Sum of revenue by region".

#	Where	What happens
1	`AnalysisBuilder.tsx`	The user edits the spec. `validateSpec()` (`lib/spec.ts:31`) mirrors the server rules and disables **Run analysis** while errors exist
2	`App.tsx:109` `runAnalysis()`	Sets `busy='analyze'`, clears the old narrative, calls `api.analyze(file, spec)`
3	`lib/api.ts:45` `form()`	Builds `FormData` : `file` (the original `File`) and `spec` (`JSON.stringify`)
4	Browser → server	`POST /api/analyze` as multipart. In dev this goes through the Vite proxy; in AWS directly to the Function URL
5	`CORSMiddleware`	Checks `Origin` against the explicit list. Preflights allow only GET/POST and `Content-Type`
6	`guard_and_log` (`main.py:67`), inside CORS	Rejects with 413 if `Content-Length` exceeds 4 MB + 64 KB **before** multipart parsing. After the response: adds `nosniff` / `no-store` and logs method, path, status and ms
7	Route `analyze` (`main.py:158`)	`_parse_json_field(AnalysisSpec, spec)` : Pydantic validation, else 422 `invalid_request` with details

#	Where	What happens
8	`_load()` (`main.py:99`)	Reads at most `max_upload_bytes + 1` bytes, then `load_table()` → `build_dataset()`
9	`ingest.load_table`	Extension allow-list → empty/size checks → CSV or XLSX reader → `_build_frame()` (headers, limits)
10	`profiling.build_dataset`	Per column: missing mask → boolean? → number? → date? → category/text. Produces `raw` (strings) and `typed` frames
11	`analysis.run_analysis`	Validate → optional merge → optional dedupe → filters → KPIs → grouping → sort/truncate → chart advice
12	FastAPI	Serialises the dict to JSON. Any `AppError` becomes `{"error":{code,message}}` with its status
13	`App.tsx`	Stores `result`, sets `measureKey` to the first measure, picks `chartKind` from `result.chart.recommended`
14	`Dashboard.tsx`	Renders KPI cards, the chart card (type toggle, measure selector, notes) and the results table

---

5. Backend in depth

5.1 Application factory, middleware and error model (`main.py`)

- **`create_app(settings)`** (`main.py:46`) builds a fresh app. It takes an optional `Settings` so tests can create apps with different limits.
- It registers `app.dependency_overrides[get_settings] = lambda: settings`, so every `SettingsDep` in a route receives *this* app's settings rather than the cached global.
- OpenAPI docs (`/api/docs`, `/api/openapi.json`) are disabled when `app_env == "production"`.
- `app.state.rate_limiter` holds one `RateLimiter` per app instance.
- **Middleware order (verified empirically):**
- `CORSMiddleware` wraps the `@app.middleware("http")` guard, so guard responses (e.g. 413) and all `AppError` responses carry CORS headers.
- Unhandled exceptions (500) are produced by Starlette's outermost `ServerErrorMiddleware`, *outside* CORS, so they have **no** CORS headers. See finding F10.
- **Error model** (`errors.py`):

```
class AppError(Exception): status_code = 400; code; message     # safe to show users
class DataError(AppError): status_code = 422                     # file/analysis problems
class AIError(AppError): 502 · AIDisabledError: 503 · AIQuotaError: 429   # in ai.py
```

- **Three exception handlers:**
- `AppError` returns its own status and code.
- `RequestValidationError` returns 422 `invalid_request` with at most 5 summarised details (`_validation_details`, `main.py:38`).
- `Exception` returns 500 `internal_error` with a generic message. Only the exception *type name* is logged, never values.
- **JSON form fields:** `spec` and `options` arrive as strings inside multipart. `_parse_json_field()` (`main.py:105`) validates them with `model_validate_json` and raises `_FieldError` (a `DataError`), so malformed JSON and schema violations both become 422.
- **Synchronous routes:** routes are plain `def`. FastAPI runs them in its threadpool, so CPU-bound pandas work never blocks the event loop.

5.2 Configuration (`config.py`)

`Settings(BaseSettings)` reads `TABULAVIBE_*` environment variables, or `.env` in the working directory.

Setting	Default	Notes
`app_env`	`development`	`development` / `test` / `production`
`allowed_origins`	`http://localhost:5173,http://127.0.0.1:5173`	`.origins` splits, strips trailing `/` and **drops** `*`
`max_upload_mb`	`4.0` (0 < x ≤ 5)	`.max_upload_bytes = int(mb·1024²)` . 4 MB × 1.33 base64 ≈ 5.3 MB, under Lambda's 6 MB
`max_rows` / `max_columns` / `max_cells`	100,000 / 60 / 2,000,000	Enforced while reading
`max_groups`	100	Grouped rows returned
`ai_provider`	`none`	`none` or `gemini`
`gemini_api_key`	`None`	`SecretStr` : never printed in reprs or logs
`gemini_model`	`gemini-3.5-flash-lite`	2.5 models are restricted to existing users (as of Oct 2026)
`ai_timeout_seconds`	20 (≤ 60)	httpx timeout
`ai_requests_per_minute`	5	Per-client burst limit

`get_settings()` is `@lru_cache` d, so the environment is read once per process.

5.3 Ingestion (`ingest.py`)

Output contract: `RawTable(frame: DataFrame of str, file_type, sheet_name, notes, empty_rows_dropped)` . Every cell is a **trimmed string**, and empty cells are `""` . Types are decided later, so CSV and XLSX share exactly one inference path.

`load_table()` (`ingest.py:47`) checks, in order: 1. **Extension:** must be `.csv` or `.xlsx` . - `.xls` , `.xlsm` , `.xlsb` , `.numbers` and `.ods` each get a specific hint; anything else gets a generic message (415). - The browser's `Content-Type` is never consulted. 2. **Empty content:** rejected. 3. **Size:** byte length must be within `max_upload_bytes` (413). 4. **Dispatch** to `_read_csv` or `_read_xlsx` , then `_build_frame` .

CSV path (`_decode` `:77` , `_read_csv` `:97`):

Step	Behaviour
Zip signature	`PK\x03\x04` → 415 "looks like a spreadsheet or archive"
Binary detection	A NUL byte in the first 8 KB is accepted only with a UTF-16 BOM; otherwise 415
Decoding	`utf-8-sig` (strips the BOM). Falls back to `cp1252` (adds a note), then `latin-1`
Delimiter	`csv.Sniffer` over the first 64 KB, restricted to `, ; TAB |` . Defaults to comma; any other delimiter is noted
Parsing	`csv.reader(..., strict=False)` . The row count is checked **while iterating**, so huge files stop early. `csv.Error` becomes `malformed_csv` with the line number

XLSX path (`_read_xlsx` `:148`):

Step	Behaviour
Container check	Must start with `PK\x03\x04`, be a valid zip, and contain `[Content_Types].xml` and `xl/workbook.xml`
Macros	Any entry ending in `vbaproject.bin` → 415 `macros_not_allowed` (catches renamed `.xlsm` files)
Zip-bomb guard	At most 5,000 entries and 120 MB total uncompressed size
Loading	`openpyxl.load_workbook(read_only=True, data_only=True, keep_links=False, keep_vba=False)`. Formulas are **never evaluated**; Excel's cached values are used
Sheet	The first worksheet only (noted if there are more)
Iteration	Trailing empty cells are trimmed per row. Column and row limits are checked as rows stream in. More than 10,000 consecutive empty rows ends reading (protects against formatted-but-empty sheets)
Cell conversion (`_cell_to_str` `:130`)	`bool` → `true` / `false` (checked before `int`). `datetime` at midnight → `YYYY-MM-DD`, otherwise `YYYY-MM-DD HH:MM:SS`. Whole floats → `int` text, other floats → `repr`. NaN → `""`

Frame construction (`_build_frame` `:234`, `_clean_headers` `:210`): - **Header row:** the first non-empty row. - **Width:** the widest row. Trailing columns are dropped only if they are empty in the header *and* in every row. - **Headers:** - Whitespace is collapsed and names are cut at 120 characters. - Blank headers become `column_N`. - Case-insensitive duplicates become `name_2`, `name_3`, … - Every rename is noted. - **Data rows:** cells are trimmed. Fully empty rows are skipped; only *interior* skips are reported (trailing blank lines are ignored). Short rows are padded with `""`. - **Final limits:** no data rows → `no_data_rows`; rows, cells and columns are re-checked.

5.4 Type inference and profiling (`profiling.py`)

Data model:

```
ColumnInfo(name, inferred_type, role, missing_count, invalid_count, unique_count,
           date_format=None, invalid_examples=[], notes=[])
Dataset(raw: DataFrame[str], typed: DataFrame, columns: dict[str, ColumnInfo], source: RawTable)
```

`raw` keeps the trimmed text, which is used for duplicates, examples and exports. `typed` holds the parsed values used for maths: float64, datetime64, the nullable `boolean` dtype, or object.

Missing tokens (`MISSING_TOKENS`, case-insensitive): `"" na n/a null none nan nil - -- #n/a`.

Inference cascade (`build_dataset`, `profiling.py:136`). It works on a sample of the first 1,000 non-missing values:

```
present == 0                          → empty     (role other)
≥90% of sample in {true,false,yes,no} → boolean   (role dimension)
≥90% of sample parse as numbers       → number    (role measure, or identifier)
≥90% of sample contain a digit
   and best date format ≥90%          → date      (role time)
otherwise:
   unique ≤ 200 or unique/present ≤ 0.2 → category (role dimension, or identifier)
   else                                 → text     (role identifier if unique/present > 0.95, else other)
```

The 90% threshold (`TYPE_THRESHOLD`) is what lets a mostly numeric column with a few `TBD` values become a number column with `invalid_count = 3`, instead of collapsing to text.

Number parsing (`parse_numbers`, `:93`) is fully vectorised: 1. Strip a currency prefix but keep the sign: `^([+-]?)\s*(₹|rs\.?|inr|\$|€|£)\s*` → `\1`. 2. Strip a trailing `%`. The value stays on its scale (`12%` → 12) and a note is added. 3. Full-match against `_NUMBER`: an optional sign, then **either** grouped digits `\d{1,3}(,\d{2,3})*,\d{3}` (accepts `1,234,567` and Indian `12,34,567`; rejects `1,2`) **or** plain digits, then optional decimals and exponent. A separate digit check prevents `"."` or `"e5"` from matching. 4. Remove commas, then `pd.to_numeric(errors="coerce")`.

Date parsing: - `_best_date_format` (`:107`) tries 20 explicit `strftime` formats on the sample and keeps the one with the highest parse rate. - Because `rate > best_rate` is strict, **earlier formats win ties**. Day-first formats come before month-first, so `03/04/2024` is read as 3 April. - If the chosen format is day-first and its month-first twin also reaches 90%, the column gets an "ambiguous" note. - Mixed formats within a column are not supported: values in other formats count as invalid.

Identifier detection (`_is_identifier`, `:122`). A column is an identifier if either: - its name matches `(^|[_\s-])(id|no|code|zip|pin|pincode|phone|mobile)$`, `^id[_\s-]`, or camelCase `…Id`; or - it is numeric with at least 20 values, all unique, and they are **consecutive whole numbers** (1, 2, 3, …).

Identifiers are never suggested as measures. An earlier version flagged *any* column of unique integers; that wrongly hid real amounts, so it was narrowed to consecutive sequences.

Case/spacing variants: for category and text columns, `unique − nunique(lower + collapsed spaces)` counts values that differ from another only by case or spacing. It is reported as a note; nothing is changed.

Profile output (`build_profile`, `:315`): - **Dataset summary:** file name (cut to 200 characters), file type, sheet, rows, columns, duplicate rows (exact, on `raw`), missing cells and percent, empty rows skipped. - **Per column** (`column_profile` `:258`): - type, role, missing count and percent, invalid count with up to 3 examples (60 characters each), distinct count, date format, notes; - plus **one** of: - `numeric_stats`: count, sum, mean, median, std (ddof=1, `null` if n < 2), min, max; - `date_range`: min, max; - `top_values`: top 5. - **Issues:** - duplicates: warning; - missing values: info below 20%, warning from 20%; - invalid values: warning; - empty column: info; - each column note: info. - **Also:** a preview (50 rows, raw strings) and suggestions: - measures: every column with the measure role; - dates: every time column; - dimensions: those with ≥ 2 values, sorted so groups of 30 or fewer come first.

`_num()` (`:248`) converts numpy scalars to Python floats and NaN/∞ to `None`, so JSON never contains NaN.

Label merging (`merge_label_variants`, `:372`), opt-in only: - For each category column, values are grouped by their normalised key (lower case, spaces collapsed). - The **most frequent original spelling** becomes canonical; ties go to the alphabetically first. - The function returns a modified *copy* of `typed` and a per-column count of changed cells.

5.5 Analysis engine (`analysis.py`)

`run_analysis(ds, spec, max_groups=100)` (`:179`) runs this pipeline:

1. `_validate` (`:31`): - duplicate measures → `duplicate_measure`; - unknown columns → `unknown_column`; - sum, mean, min or max on a non-number → `incompatible_aggregation` (message suggests Count); - grouping by a number or empty column → `incompatible_grouping`; - a value filter on a non-category, or a date filter on a non-date → `invalid_filter`.
2. **Merge** label variants (optional); each changed column is noted.
3. **Remove duplicates** (optional). Duplicates are detected on `raw`, so `North` and `north` rows are *not* duplicates of each other.
4. **Filters:** combined with AND. - Value filters use `isin`. Boolean filters map `true` / `yes` to `True`. - Date ranges are `[start, end + 1 day)`, so the end date is inclusive and includes times on that day.
5. **KPIs:** each measure is computed over the filtered frame. A row count uses a series of ones. Each KPI carries `values_used`, `values_excluded` and a plain-English `definition` (`_definition` `:102`).
6. **Grouping** (`_group_keys` `:118`):

Group type	Label	Sort key
date / day	`YYYY-MM-DD`	period start
date / week	`Week of YYYY-MM-DD` (Monday start, computed as `date − weekday`)	week start
date / month	`YYYY-MM` (`to_period("M")`)	period start
date / quarter	`YYYY-Qn`	period start

Group type	Label	Sort key
date / year	`YYYY`	period start
boolean	`true` / `false`; missing → `(missing)`	first measure, descending
category / text	the value; missing → `(missing)`	first measure, descending, then label

- Rows without a valid date are **excluded from the groups**, and a note says how many. They are still counted in the KPIs.
- Grouping uses `groupby(sort=False)`, then a Python sort: chronological for dates; for categories, by the first measure's value, with `None` values last.
- Results beyond `max_groups` are truncated, with `truncated: true` and a note. 7. **Aggregation** (`_aggregate` `:82`):
- `count` = number of non-null values.
- Every other aggregation returns `None` when there are no values. This is the **division-by-zero guard**: an empty average is "no value", not 0.
- Mean is computed explicitly as `sum / n`.
- Results are rounded at 9 decimal places to remove binary noise (`0.1 + 0.2`). 8. **Chart advice** (`_chart_advice` `:143`):

Condition	`recommended`	donut allowed
no grouping	`kpi`	no ("pick a category column")
date grouping with 2+ periods	`line`	no (time is not part of a whole)
date grouping with 1 period	`bar`	no, plus a note
category grouping	`bar`	only for 1 measure, sum or count, no negatives or nulls, 2–8 groups; otherwise the reason is returned

More than 20 groups adds the note "bar chart shows the top 20".

Response shape:

```
{ "row_count": 725, "rows_analyzed": 725,
  "kpis": [{"key":"sum__revenue","label":"Sum of revenue","column":"revenue","aggregation":"sum",
            "value":1645592.43,"values_used":707,"values_excluded":18,"definition":"Total of all ..."}],
  "grouping": {"column":"region","type":"category","time_grain":null,"total_groups":7,"truncated":false},
  "rows": [{"group":"North","row_count":184,"values":{"sum__revenue":432525.52}}],
  "measures": [{"key":"sum__revenue","label":"Sum of revenue","aggregation":"sum"}],
  "chart": {"recommended":"bar","pie_allowed":false,"pie_reason":"...","notes":[]},
  "notes": [], "spec": { ...echo of the validated spec... } }
```

Measure keys are `"{aggregation}__{column}"`, or `"count__rows"` for the row count. They are stable IDs that the frontend uses to look up values and pick the chart's measure.

`filter_options()` (`:279`) lists the sorted distinct values of category columns (≤ 200 values) and `true` / `false` for booleans. They are returned with the profile so the UI can build checkbox filters.

5.6 Exports (`export.py`)

- **`safe_cell`** (`:18`) defends against formula injection, following OWASP. Text that starts with `= + - @ TAB CR LF` gets a leading `'`, which spreadsheets show as text. **Numbers bypass it**, so `-5` stays numeric.
- **`format_number`** (`:28`): plain, locale-independent text. Whole numbers below 1e15 print as integers; everything else uses `repr(float)`, the shortest string that round-trips exactly.

- `_to_csv`: CRLF line endings plus a UTF-8 BOM (`UTF8_BOM = chr(0xFEFF)`) so Excel shows `₹` correctly. Headers also pass through `safe_cell`.
- **Cleaned data** (`cleaned_rows` `:46`), per column:

Column type	Output
number	`format_number` (`₹1,200` → `1200`)
date	`YYYY-MM-DD`, or `YYYY-MM-DD HH:MM:SS` if any value in the column has a time
boolean	`true` / `false`
category / text	the value (merged if the option is on), through `safe_cell`
empty	blank
missing (any type)	blank
invalid (any type)	the **original text**, through `safe_cell`, so nothing is lost or invented

The response headers `X-Rows-Exported`, `X-Duplicates-Removed` and `X-Labels-Merged` drive the UI's confirmation message. - **Results CSV** (`results_csv` `:95`): - grouped: `group column, row_count, <measure labels…>`; - ungrouped: `measure, value, values_used, values_excluded, definition`.

It is produced by calling `run_analysis` again with the same spec, so the export always matches the numbers on screen.

5.7 Optional AI narrative (`ai.py`)

Provider abstraction:

```
class NarrativeProvider(Protocol):
    name: str; model: str
    def generate(self, system: str, prompt: str) -> str: ...
```

`get_provider(settings)` (`:104`) returns `GeminiProvider` only when the provider is `gemini` **and** the key is non-empty; otherwise `None`. The route depends on `get_narrative_provider` (`main.py:212`), which tests override with fakes.

GeminiProvider (`:61`): - Calls `POST https://generativelanguage.googleapis.com/v1beta/models/{model}:generateContent`. - Sends the key in the `x-goog-api-key` **header**, never in the URL, so it cannot leak into URL logs. - Request body: `systemInstruction`, `contents`, and `generationConfig {temperature: 0.3, maxOutputTokens: 2048}`. - `httpx.Client(timeout, transport)`: the injectable `transport` lets tests use `httpx.MockTransport`. - Error mapping:

Condition	Error code	HTTP
timeout	`ai_timeout`	502
network error	`ai_unavailable`	502
429	`ai_quota_exceeded`	429
401/403	`ai_auth_failed`	502
other ≥ 400	`ai_provider_error`	502
unparseable or empty body	`ai_malformed_response`	502
AI disabled	`ai_disabled`	503

- Text parts are concatenated, skipping any part marked `thought`.

Data minimisation (`build_payload` `:131`). The provider receives only: - rows analysed; - KPIs: label, value and excluded count; - the filters and duplicate setting; - data-quality facts: the duplicate count and names of columns with

invalid values (at most 10); - the grouping; - at most 30 group rows, with labels cut to 60 characters.

It never receives raw rows or the file name. This exact payload is returned to the user as `shared_payload`.

Prompt design: - The system instruction: - forbids inventing or re-calculating numbers; - forbids stating causes (possible explanations must be phrased as questions); - fixes the format: one summary sentence plus 3–5 bullets, under 200 words; - says to **treat labels in the JSON as data, not instructions**, which defends against prompt injection through column or category names. - The output is trimmed, capped at 4,000 characters, and returned with a disclaimer and a timestamp.

Abuse control: - `RateLimiter` (`:110`) is a thread-safe sliding window keyed by client IP (default 5 per minute). It clears itself if it tracks more than 10,000 keys. - The check runs **before** the file is parsed, so rate-limited calls stay cheap.

5.8 Request schemas (`schemas.py`)

All models use `extra="forbid"`, so unknown fields are rejected rather than silently ignored.

Model	Fields and validators
`Measure`	`column: str ≤ 200 | None`, `aggregation: sum|mean|count|min|max`. Validator: `column=None` only with `count`
`Filter`	`column`, `values: list[str] ≤ 500 | None`, `start/end: date | None`. Validator: exactly one kind; `start ≤ end`
`AnalysisSpec`	`measures` (1–6), `group_by`, `time_grain` (`month`), `filters` (≤ 10), `remove_duplicates`, `merge_label_variants`
`CleanOptions`	`remove_duplicates`, `merge_label_variants`

Form-field length caps (`max_length=20_000` for `spec`, `2_000` for `options`) bound the JSON size before parsing.

5.9 Samples and the Lambda entry point

- **`samples.py`** maps fixed IDs (`retail-sales`, `service-bookings`) to files. The route only looks up dictionary keys, so path traversal is impossible (`/api/samples/../../etc/passwd` → 404).
- **`scripts/generate_samples.py`** uses seeded `random.Random`.
- Retail: seed 42, 720 orders plus 5 duplicated rows, 18 missing revenues, 6 `n/a` unit counts, 3 `TBD` prices, 4 lower-cased regions, and a Q4 seasonal uplift.
- Bookings: seed 7, 300 rows with ratings ~1/6 missing and `dd-mm-yyyy` display format.
- **`lambda_handler.py`**: `handler = Mangum(app, lifespan="off")`.
- Mangum translates Function URL events (payload format 2.0) to ASGI and decodes `isBase64Encoded` bodies.
- It encodes binary responses; `text/*` responses are passed through as text.

---

6. API reference

Base path `/api`. Errors are always `{"error": {"code": str, "message": str, "details"?: [str]}}`.

Method	Path	Request	Success response
GET	`/health`	–	`{"status":"ok","version":"0.1.0"}`
GET	`/config`	–	`{"limits": {max_upload_mb,max_rows,max_columns,max_cells,supported_extensions},"ai": {enabled,provider,model}}`
GET	`/samples`	–	`[{id,file_name,title,description}]`

Method	Path	Request	Success response
GET	`/samples/{id}`	–	The file (`text/csv` or the xlsx media type)
POST	`/profile`	multipart `file`	Profile (§5.4) plus `filter_options`
POST	`/analyze`	`file` , `spec` (JSON string)	Analysis result (§5.5)
POST	`/export/results`	`file` , `spec`	`text/csv` attachment `tabulavibe_results.csv`
POST	`/export/cleaned`	`file` , `options` (JSON, default `{}`)	`text/csv` attachment `tabulavibe_cleaned_data.csv` plus `X-*` headers
POST	`/narrative`	`file` , `spec`	`{provider,model,text,generated_at,disclaimer,shared_payload}`

Error codes:

HTTP	Codes
404	`sample_not_found`
413	`file_too_large` , `too_many_rows` , `too_many_columns` , `too_many_cells`
415	`unsupported_file_type` , `content_mismatch` , `macros_not_allowed`
422	`empty_file` , `no_data_rows` , `malformed_csv` , `malformed_xlsx` , `invalid_request` , `unknown_column` , `incompatible_aggregation` , `incompatible_grouping` , `invalid_filter` , `duplicate_measure`
429	`ai_quota_exceeded` , `ai_rate_limited`
502	`ai_timeout` , `ai_unavailable` , `ai_auth_failed` , `ai_provider_error` , `ai_malformed_response`
503	`ai_disabled`
500	`internal_error`

Response headers on every response: `X-Content-Type-Options: nosniff` , `Cache-Control: no-store` . CORS exposes `Content-Disposition` and the three `X-*` export headers.

---

7. Frontend in depth

7.1 State machine (`App.tsx`)

A single component owns all state; there is no router and no global store. That is deliberate: the workflow is linear and the state is small.

State	Purpose
`config` , `samples` , `serverError`	Loaded once on mount (`Promise.all`). Failure shows "The server is unavailable"
`file`	The original `File` , re-sent with every request (stateless backend)
`profile` , `spec`	Set after upload. `spec` is initialised by `defaultSpec(profile)`

State	Purpose
`result`, `analysedAt`	The last analysis and its timestamp (the report's "Prepared" date, kept out of render for purity)
`narrative`, `aiError`	AI panel
`tab`	`preview` / `quality` / `analyze` / `report`. The report tab is disabled until a result exists
`chartKind`, `measureKey`	Chart type (`line` / `bar` / `pie` / `table`) and the measure being charted
`busy`	One tagged flag: `upload`, `analyze`, `export-clean`, `export-results` or `narrative`
`error`, `exportNotice`, `exportDedupe`, `exportMerge`	Feedback and export options

Flows: - **`loadFile()`** (`:71`): - Resets everything. - Performs **client-side pre-checks**: size against `config.limits` and an extension regex. These give a fast message without uploading. - Then calls `/profile` and sets `file`, `profile`, `spec` and the preview tab. - **`loadSample()`**: fetches the sample as a Blob, wraps it in a `File`, then calls `loadFile()`. Samples go through *exactly* the same path as user files. - **Exports and narrative** send `result.spec`, the spec *that produced the visible result*, not the spec currently being edited. Downloads therefore always match the screen.

7.2 API client (`lib/api.ts`)

- `BASE_URL = VITE_API_BASE_URL ?? ''`. It is empty in development, where same-origin requests go through the proxy.
- `request()` turns failures into `ApiError(code, message, status, details)`:
- network failure → `network_error`;
- non-OK responses → the JSON error body is parsed if present, otherwise "HTTP n".
- `form(file, fields)` puts each field into `FormData` as JSON.
- Download helpers read the filename from `Content-Disposition`, and `saveBlob()` uses a temporary object URL and anchor.

7.3 Components

Component	Responsibility
`UploadPanel`	Drag-and-drop zone, a hidden labelled `<input type=file accept=…>`, the limits text, a privacy notice and sample cards
`DataPreview`	Sticky-header scrollable table (focusable, `aria-label`), type badges, `empty` placeholders
`QualityReport`	Four stat tiles; warnings and info as `Alert`s with the explanation "nothing has been changed"; a column table with stats, ranges or top values
`AnalysisBuilder`	Measure rows (column + aggregation, limited by `aggregationsFor`), group-by and time-period selects, filter cards (checkbox list or date range), dedupe and merge toggles, validation alert, submit
`Dashboard`	Summary line, notes, `KpiCards`, chart card (type toggle as `aria-pressed` buttons; donut disabled with a `title` and a "Why?" disclosure; measure select), `ResultsTable`
`Charts`	`TrendChart` (line), `BarComparison` (horizontal bars, top 20), `ShareChart` (donut with an HTML legend and percentages)
`AiNarrative`	Disabled explanation, consent text, error, "AI-generated" badge, disclaimer, and a `<details>` showing the exact `shared_payload`
`ReportView`	Print layout: title, source, date, scope (rows, dedupe, merge, filters, notes), key figures, chart, table, commentary, data quality, footer

Component	Responsibility
`ui.tsx`	`Button` (variants, `busy` spinner, `aria-busy`), `Card`, `Alert` (`role=alert` for errors, otherwise `status`; icon plus text), `Spinner`, `Badge`, `Stat`, shared class strings

7.4 Client-side logic (`lib/spec.ts`, `lib/format.ts`, `lib/charts.ts`)

- `defaultSpec`:
- measures: the sum of the first suggested measure plus a row count (or just the row count if there are no measures);
- group by: the first date column, else the first dimension;
- time period: month.
- `validateSpec` mirrors the server rules for instant feedback: duplicate measure, non-numeric aggregation, empty value filter, date filter with no bounds, start after end. The server still validates everything independently.
- **Formatting** uses `Intl.NumberFormat('en-IN')` (`LOCALE` is the single switch):
- full values have ≤ 2 decimals;
- compact values (axis ticks and bar labels) appear from 1,000 up, with ≤ 2 decimals, e.g. `4.04L`;
- `formatNumber(null)` → `–`.
- `chartTitle` builds titles such as "Sum of revenue by month (order_date)". `SERIES` holds the eight categorical colour variables.

---

8. Design system and charts

> **UI v2 (0.2.0)** replaced the v1 look. The authoritative spec is `DESIGN.md`. In summary: - Inter (self-hosted), indigo→violet brand gradient, lucide icons, light/dark/system theme toggle (`lib/theme.ts`). - A full-width top stepper (4-column grid on phones); a landing hero with drag-and-drop. - A **data-health ring** and per-column completeness meters (server fields `valid_cell_pct`, `invalid_cell_count`, `valid_pct`). - **Suggested analyses** (`lib/spec.ts` → `suggestedAnalyses`) that run in one click via `App.applySuggestion`. - Switch toggles that are styled native checkboxes; skeleton loaders. - Area-wash trend charts; the donut shows the server KPI total in its centre.
>
> The token and chart rules below still apply.

Tokens (`index.css`): - Colours are CSS custom properties on `:root`: page, surface, ink levels, lines, accent, good/warn/bad, and `--series-1..8`. - They are mapped into Tailwind v4 with `@theme inline`, so classes like `bg-surface` and `text-ink-2` resolve to variables. - Dark mode redefines the same tokens under `@media (prefers-color-scheme: dark)`, guarded by `:root:not([data-theme='light'])`. - Print rules force a white page and black ink, set `@page` margins, and use `.print-avoid-break`. App chrome uses `print:hidden`.

Chart rules: - Colours come from a validated categorical palette applied in fixed order. - One measure per chart: never two y-axes. Users switch measures with a selector instead. - Lines are 2 px with 8 px dots and 2 px surface-coloured rings. - Bars are at most 24 px thick, have a 4 px rounded end, are horizontal for readable labels, and are sorted. - Gridlines are hairline and recessive. Tooltips sit on every chart. - Each chart is wrapped in `role="img"` with an `aria-label`, and a data table is always one click away. - Text uses ink colours, never series colours. The donut legend is HTML with a swatch and a percentage. - Animations are disabled (`isAnimationActive={false}`) for stability and print.

Accessibility: - Every form control has a `<label>`. - Focus is visible (`:focus-visible` outline). - Tables have `<caption class=sr-only>` and `scope` attributes, and scroll containers are focusable. - Errors use `role=alert`. - Status meaning never relies on colour alone: there is always an icon and text.

---

9. Testing strategy

Layer	Tool	Count	What is covered
Ingestion	pytest	21 fns	CSV: delimiters, BOM, cp1252, headers, empty rows, padding. XLSX: native types, uncached formulas, multi-sheet. Rejections: 6 bad extensions, content mismatch both ways, binary, macros, random zip, empty, header-only, size, rows, columns, cells
Profiling	pytest	10	Number parser table (₹, $, Indian grouping, `1,2`, `%`, `1e3`), fixture types, stats, day-first and ambiguity notes, booleans, identifiers, empty column, case variants
Analysis	pytest	14 fns (more cases via parametrize)	Each aggregation vs hand maths, category sort, donut rules, dedupe, month/quarter/year/week grouping, filters, empty average, no matches, precision, 7 invalid specs, 6 malformed specs, truncation, opt-in merge
API	pytest + TestClient	21	Health/config/samples, path traversal, profile CSV/XLSX, untrusted content type, 413 pre-parse guard, 422 shapes, analyze, export contents and injection safety, CORS allow/deny, `*` ignored, docs hidden in prod
AI	pytest + fakes + `httpx.MockTransport`	12 fns	Disabled by default, key never leaked, aggregates-only payload, quota/provider/empty errors, rate limit, core features unaffected, Gemini request shape (header key, not URL), 6 error mappings, timeout
Lambda	pytest	4	Real Mangum handler with Function URL v2 events and base64 multipart bodies
Cross-check	pytest	1	Sample totals per region vs an independent `csv`-module computation
Frontend units	Vitest + Testing Library (jsdom)	15	Formatting, spec defaults and validation, API error mapping, FormData shape, the full App journey with mocked fetch, client rejection, server errors, offline, AI failure isolation, disabled submit
End-to-end	Playwright (system Chrome, desktop + Pixel 7)	3 × 2	Real servers. Full journey with real CSV downloads checked line by line, Excel line chart and donut reason, three invalid uploads

Fixture philosophy: `tests/conftest.py` holds a 10-row CSV whose expected values are written as comments and were computed by hand: sums, means, groups, the duplicate row, the invalid date, and formula cells. Tests assert those numbers, not values copied from the implementation.

Infrastructure checks: - `cfn-lint` validates the template, including the SAM transform. - The Lambda package was built and invoked inside AWS's official arm64 image with the Runtime Interface Emulator. That was a one-off verification, not an automated test.

---

10. Build, tooling and developer workflow

Area	Tool	Config
Python lint/format	ruff (`E F W I B UP S SIM`; ignores `S101`, `B008`, `E501`)	`backend/pyproject.toml`
TypeScript	`tsc -b` (strict-style flags: unused locals/params, `verbatimModuleSyntax`, `erasableSyntaxOnly`)	`tsconfig.app.json`
JS lint	oxlint (react, typescript, oxc plugins)	`.oxlintrc.json`
Bundler	Vite 8 with `@vitejs/plugin-react` and `@tailwindcss/vite`; `optimizeDeps.include: ['recharts']`	`vite.config.ts`

Area	Tool	Config
Unit tests	Vitest (jsdom, `src/test/setup.ts` adds jest-dom, cleanup and a ResizeObserver stub)	`vite.config.ts` `test` block
E2E	Playwright: `webServer` starts uvicorn and vite on 8011/5181, `channel: 'chrome'`	`playwright.config.ts`
Infrastructure	cfn-lint 1.58, SAM CLI (not installed locally)	`infra/`
Audits	`npm audit`, `pip-audit`	–
Line endings	`.gitattributes`: `* text=auto eol=lf`; xlsx and png marked binary	–

Production frontend build: `tsc -b && vite build` → `frontend/dist`. The output is about 655 kB of JS (192 kB gzipped) and 19 kB of CSS. It is a single chunk; Vite warns because it is over 500 kB.

---

11. Cloud architecture (AWS)

`infra/template.yaml` (SAM) declares the following resources.

Resource	Key properties	Why
`ApiFunction` (`AWS::Serverless::Function`)	python3.14, **arm64**, 1024 MB, 20 s timeout, handler `app.lambda_handler.handler`, `CodeUri: build/api/`. Environment: `APP_ENV=production`, `ALLOWED_ORIGINS=https://<cloudfront domain>`, AI settings. `FunctionUrlConfig.AuthType: NONE`	arm64 is about 20% cheaper per GB-s; the Function URL has no extra charge
`ApiLogGroup`	`/aws/lambda/<stack>-api`, retention 7 days (parameter)	Caps log storage
`FrontendBucket`	All public access blocked, `BucketOwnerEnforced`, SSE-S3	Private origin
`FrontendBucketPolicy`	`s3:GetObject` only for `cloudfront.amazonaws.com` with `AWS:SourceArn` = this distribution; deny without TLS	Least privilege
`FrontendOac`	Origin Access Control, sigv4, always sign	Replaces the legacy OAI
`FrontendDistribution`	HTTP/2+3, `PriceClass_200` (includes India), `index.html` default object, redirect to HTTPS, GET/HEAD, compression, managed CachingOptimized and SecurityHeaders policies	CDN, TLS and security headers
`CostBudget` (conditional on `AlertEmail`)	Monthly cost budget; emails at 50% actual, 100% actual and 100% forecast	Budgets with email alerts are free

Parameters: `AlertEmail`, `MonthlyBudgetUsd` (default 1), `AiProvider` (`none`), `GeminiApiKey` (`NoEcho`), `GeminiModel`, `LogRetentionDays`. **Outputs:** `FrontendUrl`, `ApiUrl`, `FrontendBucketName`, `DistributionId`.

Packaging: - `stage_backend.py` copies only `app/` and `requirements.txt` into `infra/build/api`, so tests and virtual environments never ship. - `sam build --use-container` installs Linux arm64 wheels. - Measured package size: 152.5 MB unzipped (limit 250 MB) and 46.5 MB zipped.

`deploy.ps1` **pipeline:** 1. Check tools and run `sts get-caller-identity`. 2. Run the backend tests. 3. Stage, then `sam build`. 4. `sam deploy --resolve-s3` with parameters. 5. Read the stack outputs. 6. Build the frontend with `VITE_API_BASE_URL`. 7. Sync to S3: hashed assets as `immutable`, `index.html` as `no-cache`. 8. Invalidate `/index.html`. 9. Smoke-test `/health`, `/config`, `/samples` and the site.

Cost posture (details in `DEPLOYMENT.md`): - Lambda (1M requests, 400k GB-s), CloudFront (1 TB, 10M requests) and CloudWatch Logs (5 GB) have always-free tiers. - S3 storage likely does not, which costs paise per month. - The public

Function URL is the main cost risk.

12. Security model

Threat	Mitigation	Code
Disguised or malicious files	Extension allow-list plus content signatures; XLSX container validation; `vbaProject.bin` rejection	`ingest.py:47` , `:148`
Formula execution	openpyxl `data_only=True` , `read_only` , `keep_links=False` ; no evaluation	`ingest.py`
Zip bombs	At most 5,000 entries and 120 MB uncompressed	`ingest.py`
Memory or CPU exhaustion	Content-Length guard before parsing; capped read; row, column and cell limits while streaming; empty-row cutoff; Lambda timeout	`main.py:67` , `ingest.py`
CSV / formula injection in exports	`safe_cell` prefix `'` ; headers included	`export.py:18`
Cross-origin abuse from browsers	Explicit origin list, `*` dropped, no credentials, GET/POST only	`config.py` , `main.py`
Information disclosure	Generic 500 responses; summarised validation details; docs off in production; logs record only method, path, status and time	`main.py`
Secret leakage	`SecretStr` ; key sent in a header; `/config` never returns it; `.env` gitignored; only `VITE_API_BASE_URL` reaches the browser	`config.py` , `ai.py`
Sending data to a third party	Aggregates only; payload shown to the user; free-tier data-use notice	`ai.py:131`
Prompt injection via labels	System instruction treats JSON labels as data; output length capped; output is plain text (rendered with `whitespace-pre-line` , never as HTML)	`ai.py` , `AiNarrative.tsx`
XSS	React escapes all text; no `dangerouslySetInnerHTML` anywhere	frontend
Path traversal	Samples come from a fixed dictionary	`main.py:144`
Cloud access	Private bucket with OAC and SourceArn; Lambda role limited to logs; HTTPS only	`template.yaml`

Residual risks: - The Function URL is unauthenticated. CORS is a browser control, not access control. - The rate limiter is per instance and in memory. - A Gemini key would sit in a Lambda environment variable, readable by anyone with console access.

13. Performance and limits

- **Complexity:** parsing is O(cells). Inference samples 1,000 values per column for type detection, then runs one vectorised pass over the whole column. Date detection costs 20 × `to_datetime` on the sample, per column.
- **Measured locally** (TestClient on a dev PC): retail sample profile 0.17 s; Excel sample profile 0.68 s, analyze 0.26 s.
- **Lambda:**
- The emulated arm64 timings (22 s cold, 2–9 s warm) came from QEMU emulation and are **not representative**.
- Real cold starts are dominated by importing pandas and numpy (≈150 MB package), probably a few seconds, but unmeasured.
- **Re-upload cost:** at most 4 MB per request. The frontend re-sends the file for analyze, export and narrative.
- **Bundle:** 192 kB gzipped. Recharts is the largest dependency.

14. Findings from the reverse-engineering pass

Behaviours noticed while re-reading the code for this document. None breaks the tested user journeys, but each is worth knowing.

#	Finding	Impact	Suggested fix
F1	The backend allows grouping by **text** columns (`GROUPABLE_TYPES` includes `text`), but the UI only offers category, yes/no and date (`groupableColumns`)	API clients can create groups with many members; the UI is stricter. Harmless	Align the two, or document that the API is more permissive
F2	`filter_options` come from the **un-merged** values. With **Merge labels** on, a filter that ticks only `north` matches nothing, because `north` has been merged into `North`	Confusing edge case	Return options after merging, or apply filters before merging
F3	Grouped results exclude rows with invalid dates, while the KPI cards include them, so group totals can be lower than the KPI total	A note explains it, but users may still wonder	Add an "unassigned" row or show the excluded total
F4	The `freq` mapping in `_group_keys` contains `"W-MON"`, which is never used, because weeks are computed manually	Dead value only	Remove it
F5	The CSV row-limit check counts blank lines towards `max_rows + 1` while streaming	A file with many blank lines can be rejected slightly early	Count only non-empty rows (as the XLSX path does)
F6	The rate limiter keys on `request.client.host`. On Lambda via Mangum this is the Function URL `sourceIp`; behind a proxy it would be the proxy's IP	All users could share one bucket if fronted by a proxy	Read `X-Forwarded-For` from a trusted proxy
F7	`_CURRENCY_PREFIX` strips a leading `rs` / `inr` from any value before the number check	Text like `rs500` becomes 500; text like `rsvp` still fails as a number. Acceptable	Require a word boundary or a dot/space after `rs`
F8	`Cache-Control: no-store` is added to **every** response, including sample downloads	Samples are re-downloaded each time (tiny)	Allow caching on `/api/samples/*`
F9	`merge_label_variants` only processes **category** columns. Text columns with case variants get the note but are never merged	Rare: high-cardinality text is seldom grouped	Document it, or include text columns
F10	**Verified by test:** unexpected 500 responses come from Starlette's outermost `ServerErrorMiddleware` and carry **no CORS headers**. 4xx responses do	In the AWS cross-origin setup the browser hides the 500 body, so users see "Could not reach the server" instead of "Something went wrong". Not visible in dev, where the proxy makes it same-origin	Catch `Exception` in the `guard_and_log` middleware (inside CORS) and return the JSON 500 there, or add the CORS headers in the 500 handler

15. How to extend the system

Add an aggregation (e.g. median): 1. Add it to `Aggregation` in `schemas.py`. 2. Add it to `NUMERIC_AGGS` and `AGG_LABELS`, and handle it in `_aggregate` and `_definition`, in `analysis.py`. 3. Decide whether a donut makes sense for it (`_chart_advice`). 4. Frontend: add it to the `Aggregation` type, `NUMERIC_AGGREGATIONS` and `AGG_LABELS`. 5. Tests: a hand-computed case in `test_analysis.py`, plus extending the malformed-spec cases.

Add a file format (e.g. `.ods`): 1. Write a reader that returns `list[list[str]]` with the same limit checks. 2. Register its extension in `load_table`. 3. Accept it in the frontend regex, the `accept` attribute and the limits text. 4. Add rejection

and acceptance tests.

Inference, analysis and exports need no changes, because everything converges on `RawTable`.

Add an AI provider (e.g. a local Ollama model): 1. Implement `NarrativeProvider` (`name`, `model`, `generate(system, prompt)`) and map its errors onto `AIError` codes. 2. Add it to the `ai_provider` Literal and `get_provider`. 3. Add a template parameter value if it will run on AWS. 4. Test it with `httpx.MockTransport` as in `test_ai.py`.

Saved report templates (the recommended next feature): - An `AnalysisSpec` is already a self-contained JSON document, and `result.spec` echoes it back. - Export or import it as a file in the UI, and validate it against the new profile with `validateSpec`. No backend change is required.

---

16. Design decisions and trade-offs

Decision	Alternatives considered	Why this one
Stateless re-upload	Server sessions, S3 upload storage, a database	Zero storage cost and privacy by design; fits Lambda. Cost: re-parsing per request
Strings first, then infer	Let pandas infer while reading	One rule set for CSV and XLSX; precise control over currency, Indian grouping and ambiguity; invalid values can be reported
90% type threshold	Strict 100%	A few typos shouldn't turn a revenue column into text
Day-first date tie-break	Month-first, or ask the user	The target users are Indian. The choice is surfaced as a note
Report, don't fix	Auto-impute, auto-dedupe	Trust: every change is visible and opt-in
Python-only maths	Compute in JS, let the LLM compute	One audited implementation; the LLM can't hallucinate numbers
Mangum	Lambda Web Adapter	No second process, smaller cold start, simpler zip
Function URL	API Gateway HTTP API	No extra cost or service. Trade-off: no built-in throttling or auth
App-level CORS	Function URL CORS	One configuration works locally and in AWS; avoids duplicate headers
Gemini over raw REST	Google SDK	No dependency; trivially mockable; key in a header
Single-component state	Redux, Zustand, a router	Small linear workflow; fewer moving parts
System-Chrome Playwright	Bundled browsers	No 150 MB download; tests what users run

---

17. Interview talking points

Pitch (30 seconds): "TabulaVibe AI turns messy small-business spreadsheets into trustworthy dashboards. I designed it so Python is the only source of numbers: pandas computes every KPI with explicit definitions, the UI just formats, and the optional Gemini layer only narrates pre-computed aggregates. It never sees raw rows. The backend is stateless, so it runs on a single Lambda function with no database, and it's covered by 108 backend tests, 15 frontend tests and Playwright journeys in real Chrome."

Deep-dive questions you can answer from this document:

Question	Answer
How do you detect column types?	§5.4: missing tokens, a boolean → number → date → category/text cascade with a 90% threshold, vectorised regex number parsing including Indian grouping, a date-format tournament with a day-first tie-break
How do you keep the LLM honest?	§5.7: aggregates-only payload; a system prompt banning new numbers and causal claims; labels treated as data; output capped and labelled; payload shown to the user
What happens with a malicious file?	§5.3 and §12: signatures, macro detection, zip-bomb caps, no formula evaluation, limits while streaming
How do you test calculations?	§9: hand-computed fixtures plus an independent `csv`-module cross-check, rather than snapshots of the implementation's output
Why no database?	§2.3 and §16: stateless re-upload means zero storage, privacy by design and Lambda-friendliness
How would you scale it?	Lambda scales horizontally already. Next steps: CloudFront + WAF rate limits in front of the API, code-splitting, and caching parsed datasets by content hash if re-parsing ever shows up in profiles
Biggest bug you hit?	Identifier detection flagged any column of unique integers (it hid real amounts), so it was narrowed to consecutive sequences. Also a formatter rewriting `U+FEFF` escapes, fixed with `chr(0xFEFF)`
What would you do next?	Saved report templates (§15), period-over-period KPI deltas, multi-sheet support, and fixes F1–F10 (§14), starting with F10 (500s without CORS headers) before any AWS deployment

18. Glossary

Term	Meaning here
Measure	A numeric column you aggregate (e.g. revenue)
Dimension	A categorical column you group by (e.g. region)
Identifier	An ID-like column, never suggested as a measure
Missing	Empty or a placeholder token. Excluded and counted
Invalid	Present, but doesn't fit the column type (e.g. `TBD` in a number column). Excluded, counted and kept as the original text in exports
Spec	The `AnalysisSpec` JSON describing one analysis
Measure key	`aggregation__column`, the stable ID of a measure in results
Time grain	day / week / month / quarter / year bucketing for date grouping
OAC	CloudFront Origin Access Control, which lets only CloudFront read the private S3 bucket
Function URL	A built-in HTTPS endpoint for a Lambda function (payload format 2.0)
Mangum	An adapter that runs an ASGI app (FastAPI) inside Lambda
Formula injection	Spreadsheet cells beginning with `=`, `+`, `-` or `@` executing as formulas when a CSV is opened